

Pre-processing_FeatureEngineering_FS_200_Modeling

November 11, 2025

```
[2]: !pip install emoji
import pandas as pd

from google.colab import drive
drive.mount('/content/drive')

# read sample data from csv
file_path = '/content/drive/My Drive/notebooks/Yelp JSON/yelp_dataset (Unzipped_
↳Files)/yelp_review_data_binned.csv'
df_review_data_binned = pd.read_csv(file_path)

# Null value checking
print(df_review_data_binned.isnull().sum())
```

Collecting emoji

Downloading emoji-2.15.0-py3-none-any.whl.metadata (5.7 kB)

Downloading emoji-2.15.0-py3-none-any.whl (608 kB)

0.0/608.4 kB

? eta -:--:--

608.4/608.4 kB

28.3 MB/s eta 0:00:00

Installing collected packages: emoji

Successfully installed emoji-2.15.0

Mounted at /content/drive

review_id	0
user_id	0
business_id	0
stars	0
useful	0
funny	0
cool	0
text	0
date	0
stars_business	0
stars_binned	0
dtype: int64	

```

[3]: # Pre-processing
import emoji
import re
import nltk
import unicodedata
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
df_review_data_binned = df_review_data_binned.dropna(subset=['stars_binned'])
print(df_review_data_binned.shape)

# Identify if there are any emoji characters

def contains_emoji(text):
    if text is None:
        return False

    return emoji.emoji_count(text) > 0

def clean_text(text):
    if text is None:
        return ""
    text = emoji.replace_emoji(text, replace='')
    # convert to lower case
    text = text.lower()
    # remove strings that start with http
    text = re.sub(r'http\S+', '', text)
    # remove HTML tags
    text = re.sub(r'<.*?>', '', text)
    # remove special characters
    text = re.sub(r'www\S+', '', text)
    # remove punctuations
    text = re.sub(r'[\w\s]', '', text)
    # remove numbers
    text = re.sub(r'\d+', '', text)
    text = unicodedata.normalize('NFKD', text)
    text = text.encode('ascii', 'ignore').decode('utf-8')
    #remove any left non-ascii characters
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    # remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()

    return text

df_review_data_binned["text"] = df_review_data_binned["text"].apply(clean_text)
# download stopwords
nltk.download('stopwords')
# download tokenizer data

```

```

nltk.download('punkt')
# remove stopwords
stop_words = set(stopwords.words('english'))
# remove stopwords from text column
df_review_data_binned['text'] = df_review_data_binned['text'].apply(lambda x: '␣'
    ↪'.join([word for word in x.split() if word not in (stop_words)]))
# perform lemitazation
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
# download lexical database
nltk.download('wordnet')
# multilingual lexical database
nltk.download('omw-1.4')
# perform lemmatization
df_review_data_binned['text'] = df_review_data_binned['text'].apply(lambda x: '␣'
    ↪'.join([lemmatizer.lemmatize(word) for word in x.split()]))

```

(411589, 11)

```

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...

```

```

[4]: # TfidfVectorizer (from sklearn.feature_extraction.text) converts raw text into
    ↪numerical feature vectors using the TF-IDF weighting scheme.
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(
    min_df=5, # ignore words that appear in < 5 documents,
    max_df=0.8, # ignore words that appear in > 80% of the documents,
    max_features=10000,
    ngram_range=(1, 2))
corpus = df_review_data_binned['text']

X = vectorizer.fit_transform(corpus)
print(type(X))
print(X.shape)

```

```

<class 'scipy.sparse._csr.csr_matrix'>
(411589, 10000)

```

```

[5]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
# Plot heatmap of top 20 features
#Sum TF-IDF scores for each feature across all documents

```

```

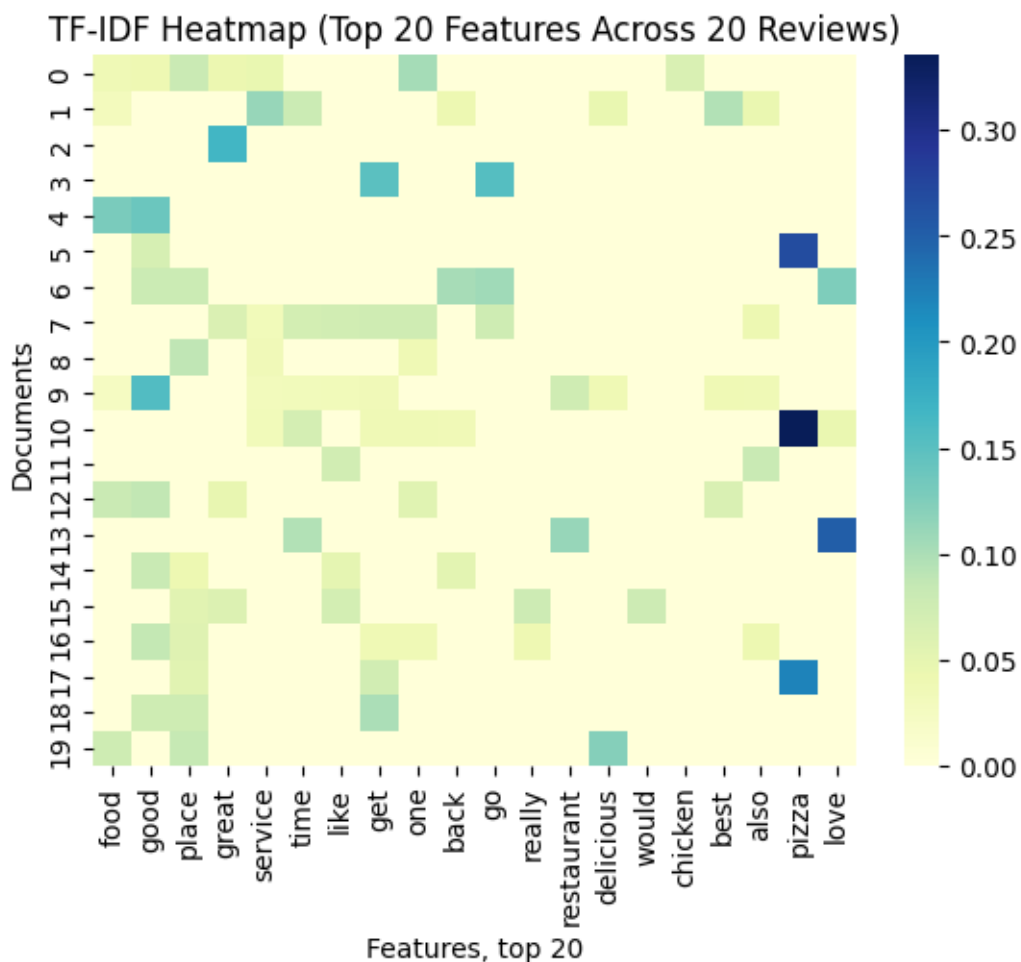
feature_sums = np.array(X.sum(axis=0)).flatten()
feature_names = np.array(vectorizer.get_feature_names_out())

# Create a pandas Series and sort
top_features = pd.Series(feature_sums, index=feature_names).
    ↪sort_values(ascending=False).head(20)

# Select top features to plot
top_feature_indices = [vectorizer.vocabulary_[f] for f in top_features.index]
X_top = X[:, top_feature_indices].toarray()[:20] # first 20 docs

# Plot heatmap
sns.heatmap(X_top, cmap="YlGnBu", xticklabels=top_features.index)
plt.title("TF-IDF Heatmap (Top 20 Features Across 20 Reviews)")
plt.xlabel("Features, top 20")
plt.ylabel("Documents")
plt.show()

```



```
[6]: from sklearn.model_selection import train_test_split
from sklearn.feature_selection import SelectKBest, chi2

# Define target variable (ensure it's discrete)
y = df_review_data_binned['stars_binned']
y = pd.Series(y).astype('category').cat.codes

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# number of features to select from the data sample
k = 1000 if X_train.shape[1] > 1000 else X_train.shape[1]

# Mutual Information feature selection
selector = SelectKBest(score_func=chi2, k=k)
X_train_chi2_selection = selector.fit_transform(X_train, y_train)
X_test_chi2_selection = selector.transform(X_test)
# selected features
selected_features = feature_names[selector.get_support()]
print(f"Selected features (k): {X_train_chi2_selection.shape[1]}")
print(f"Shapes -> Train: {X_train_chi2_selection.shape}, Test: ␣
↳{X_test_chi2_selection.shape}")
```

Selected features (k): 1000

Shapes -> Train: (329271, 1000), Test: (82318, 1000)

```
[8]: from wordcloud import WordCloud

selected_text = " ".join(selected_features)

wordcloud = WordCloud(
    width=1200,
    height=600,
    background_color="white",
    colormap="viridis",
    max_words=500,
    random_state=42
).generate(selected_text)

# Plot
plt.figure(figsize=(9, 6))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.title("WordCloud of Chi-Square Selected Features (Top 1000)", fontsize=16)
plt.show()
```

```

from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.model_selection import train_test_split
import time

def batched_forward_selection(
    X,
    y,
    n_features_to_select=300,
    batch_size=20,
    val_size=0.2,
    random_state=42,
):
    """Perform batched forward feature selection using Logistic Regression.

    Parameters
    -----
    X : train feature matrix (n_samples, n_features)

    y : target(n_samples)

    n_features_to_select : int, default=300
    Number of features to select.

    batch size : int, default=20
    """

```

```

Number of features to add per iteration.

val_size : float, default=0.2
breakdown of data in the validation split.

random_state : int, default=42
random state for reproducibility.

Returns
-----
selected : list of int
Indices of the selected features.
"""
X_train_batch, X_val_batch, y_train_batch, y_val_batch = train_test_split(
    X, y, test_size=val_size, random_state=random_state, stratify=y
)
base_estimator = LogisticRegression(
    max_iter=800,
    solver="saga" # used for multiclass,
)

n_samples, n_features = X_train_batch.shape
selected = []
remaining = list(range(n_features))

while len(selected) < n_features_to_select and remaining:

    scores_this_round = []
    for feat in remaining:
        candidate_feats = selected + [feat]
        X_tr = X_train_batch[:, candidate_feats]
        X_va = X_val_batch[:, candidate_feats]

        base_estimator.fit(X_tr, y_train_batch)
        y_pred = base_estimator.predict(X_va)
        # accuracy score is used for performance gain
        score = accuracy_score(y_val_batch, y_pred)
        scores_this_round.append((feat, score))

    # pick best batch
    scores_this_round.sort(key=lambda x: x[1], reverse=True)
    best_batch = []
    for feat, score in scores_this_round[:batch_size]:
        best_batch.append(feat)

    selected.extend(best_batch)

```

```

        remaining = [f for f in remaining if f not in best_batch]
        # print at what time features selected
        print(f'Selected at {time.strftime("%Y-%m-%d %H:%M:%S")}: {len(selected)}\n
↪/ {n_features_to_select}')
        return selected[:n_features_to_select]

```

```

[18]: selected_idx = batched_forward_selection(
        X_train_chi2_selection,
        y_train,
        n_features_to_select=300, # change if needed
        batch_size=20,
        val_size=0.2,
        random_state=42,
    )

```

```

Selected at 2025-11-10 06:56:30: 20 / 200
Selected at 2025-11-10 07:38:02: 40 / 200
Selected at 2025-11-10 08:26:06: 60 / 200
Selected at 2025-11-10 09:18:14: 80 / 200
Selected at 2025-11-10 10:14:34: 100 / 200
Selected at 2025-11-10 11:14:35: 120 / 200
Selected at 2025-11-10 12:15:49: 140 / 200
Selected at 2025-11-10 13:18:22: 160 / 200
Selected at 2025-11-10 14:21:05: 180 / 200
Selected at 2025-11-10 15:24:08: 200 / 200

```

```

[19]: # adjust train amd test dataset based on forward selection
X_train_after_fs = X_train_chi2_selection[:, selected_idx]
X_test_after_fs = X_test_chi2_selection[:, selected_idx]

```

```

[20]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
# Train classifier (Logistic Regression)
logistic_regression = LogisticRegression(max_iter=2000,
                                         n_jobs=-1,
                                         solver="saga",
                                         class_weight = 'balanced',
                                         multi_class='multinomial')
logistic_regression.fit(X_train_after_fs, y_train)

y_pred = logistic_regression.predict(X_test_after_fs)

print("\nEvaluation Metrics\n")
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred,
↪digits=3))

```

```

print("Macro F1-score:", f1_score(y_test, y_pred, average='macro'))
print("Weighted F1-score:", f1_score(y_test, y_pred, average='weighted'))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues")
plt.title("Confusion Matrix - Logistic Regression after FS Selection")
plt.show()

```

```

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247:
FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed
in 1.7. From then on, it will always use 'multinomial'. Leave it to its default
value to avoid this warning.

```

```
warnings.warn(
```

Evaluation Metrics

Logistic Regression Accuracy: 0.3643917490706771

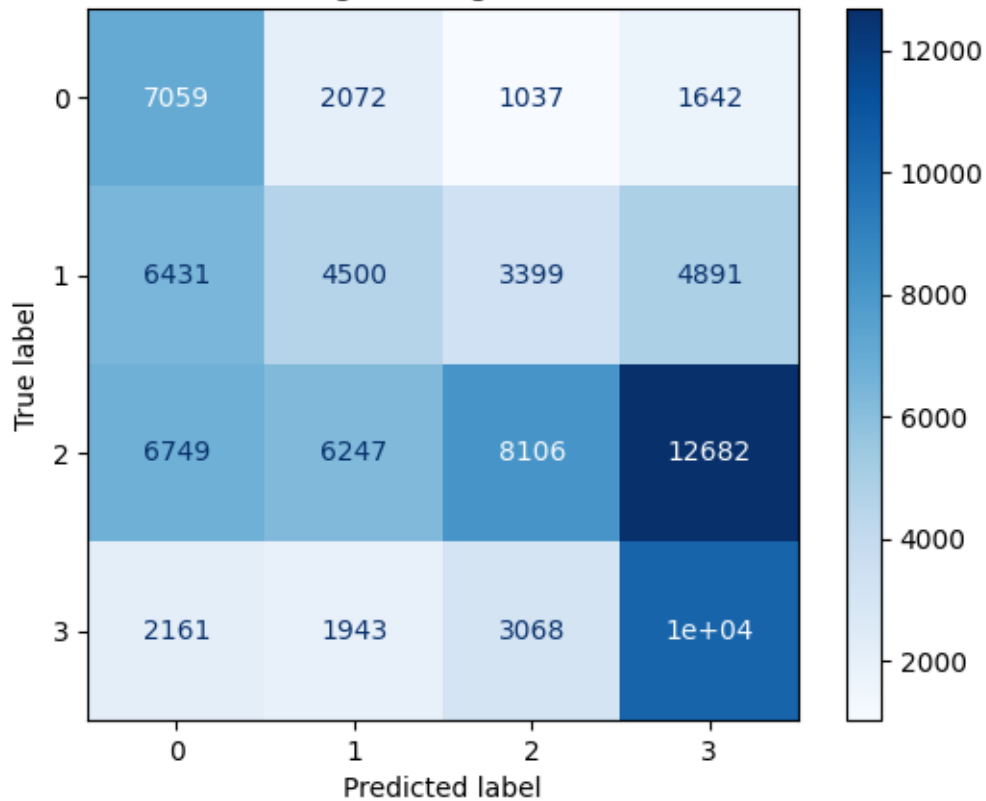
Classification Report:

	precision	recall	f1-score	support
0	0.315	0.598	0.413	11810
1	0.305	0.234	0.265	19221
2	0.519	0.240	0.328	33784
3	0.350	0.590	0.439	17503
accuracy			0.364	82318
macro avg	0.372	0.416	0.361	82318
weighted avg	0.404	0.364	0.349	82318

Macro F1-score: 0.36122545679715495

Weighted F1-score: 0.34912659189835776

Confusion Matrix - Logistic Regression after FS Selection



```
[21]: from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report, \
    confusion_matrix, ConfusionMatrixDisplay

# Naive Bayes model
nb = MultinomialNB()
nb.fit(X_train_after_fs, y_train)

# Predictions and evaluation
y_pred = nb.predict(X_test_after_fs)

print("\nEvaluation Metrics Multinomial Naive Bayes\n")
print("Multinomial Naive Bayes Accuracy:", accuracy_score(y_test, y_pred))
print("Classification report:\n", classification_report(y_test, y_pred))
print("Macro F1-score:", f1_score(y_test, y_pred, average='macro'))
print("Weighted F1-score:", f1_score(y_test, y_pred, average='weighted'))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
```

```

disp.plot(cmap="Blues")
plt.title("Confusion Matrix - Naive Bayes after FS Selection")
plt.show()

```

Evaluation Metrics Multinomial Naive Bayes

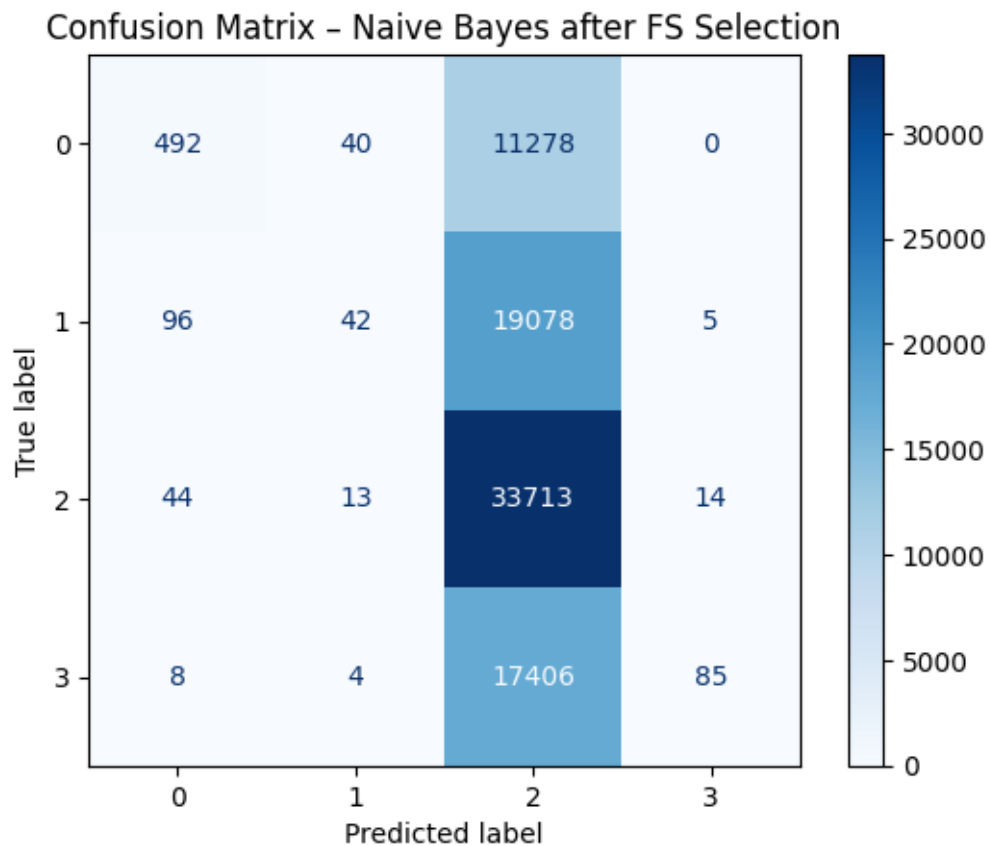
Multinomial Naive Bayes Accuracy: 0.41706552637333266

Classification report:

	precision	recall	f1-score	support
0	0.77	0.04	0.08	11810
1	0.42	0.00	0.00	19221
2	0.41	1.00	0.58	33784
3	0.82	0.00	0.01	17503
accuracy			0.42	82318
macro avg	0.61	0.26	0.17	82318
weighted avg	0.55	0.42	0.25	82318

Macro F1-score: 0.16950868830502505

Weighted F1-score: 0.25449441596027034



```
[22]: # Decision Tree Classifier

from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(
    criterion="gini",    # or "entropy"
    max_depth=20,       # keep it from overfitting on text
    min_samples_leaf=5, # smooth leaves a bit
    random_state=42
)
dt.fit(X_train_after_fs, y_train)
y_pred_dt = dt.predict(X_test_after_fs)

print("\nEvaluation Metrics Decision Tree\n")
print("\nDecision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Classification report:", classification_report(y_test, y_pred))
print("Macro F1-score:", f1_score(y_test, y_pred, average='macro'))
print("Weighted F1-score:", f1_score(y_test, y_pred, average='weighted'))

# Confusion matrix for the tree
cm = confusion_matrix(y_test, y_pred_dt)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Purples")
plt.title("Confusion Matrix - Decision Tree after FS")
plt.show()
```

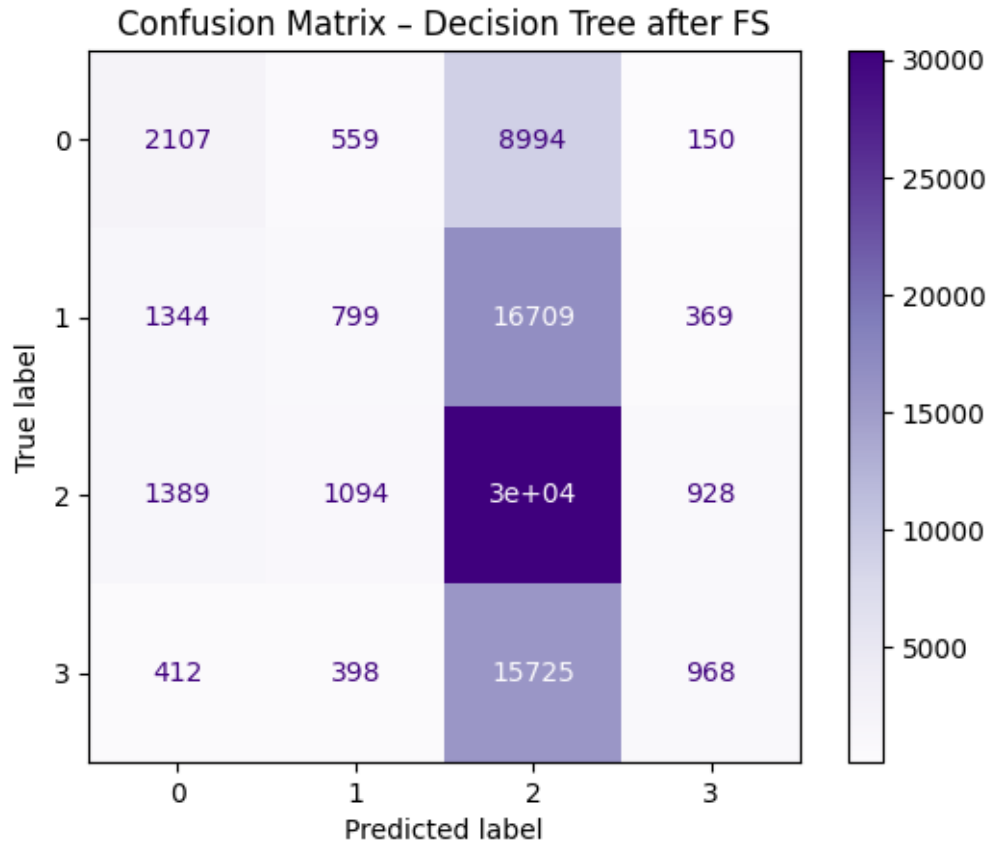
Evaluation Metrics Decision Tree

Decision Tree Accuracy: 0.4160329454068369

Classification report:		precision	recall	f1-score	support
0	0.77	0.04	0.08		11810
1	0.42	0.00	0.00		19221
2	0.41	1.00	0.58		33784
3	0.82	0.00	0.01		17503
accuracy			0.42		82318
macro avg	0.61	0.26	0.17		82318
weighted avg	0.55	0.42	0.25		82318

Macro F1-score: 0.16950868830502505

Weighted F1-score: 0.25449441596027034



```
[23]: # Random forest classifier
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(
    n_estimators=200,
    class_weight = 'balanced',
    max_depth=None,
    n_jobs=-1,
    random_state=42
)
rf.fit(X_train_after_fs, y_train)
y_pred_rf = rf.predict(X_test_after_fs)
print("\nEvaluation Metrics Random Forest\n")
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print("Classification report:\n", classification_report(y_test, y_pred))
print("Macro F1-score:", f1_score(y_test, y_pred, average='macro'))
print("Weighted F1-score:", f1_score(y_test, y_pred, average='weighted'))

# optional: confusion matrix for RF
cm = confusion_matrix(y_test, y_pred_rf)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
```

```

disp.plot(cmap="Greens")
plt.title("Confusion Matrix - Random Forest after FS")
plt.show()

```

Evaluation Metrics Random Forest

Random Forest Accuracy: 0.41648242182754686

Classification report:

	precision	recall	f1-score	support
0	0.77	0.04	0.08	11810
1	0.42	0.00	0.00	19221
2	0.41	1.00	0.58	33784
3	0.82	0.00	0.01	17503
accuracy			0.42	82318
macro avg	0.61	0.26	0.17	82318
weighted avg	0.55	0.42	0.25	82318

Macro F1-score: 0.16950868830502505

Weighted F1-score: 0.25449441596027034

